

Practical Week 8

1. Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

```
root@DUDDUKURIs  GNU nano 8.7.1  src/dynamic_memory.c *
#include <stdio.h>
#include <stdlib.h>

int main() {
    int i;

    // malloc
    int *a = malloc(5 * sizeof(int));

    if (a == NULL) {
        printf("malloc failed\n");
        return 1;
    }

    for (i = 0; i < 5; i++)
        a[i] = (i + 1) * 10;

    printf("malloc: ");
    for (i = 0; i < 5; i++)
        printf("%d ", a[i]);
    printf("\n");

    // calloc
    int *b = calloc(5, sizeof(int));

    if (b == NULL) {
        printf("calloc failed\n");
        free(a);
        return 1;
    }

    printf("calloc: ");
    for (i = 0; i < 5; i++)
        printf("%d ", b[i]);
    printf("\n");

    // realloc
    a = realloc(a, 10 * sizeof(int));

    if (a == NULL) {
        printf("realloc failed\n");
        free(b);
        return 1;
    }

    for (i = 5; i < 10; i++)
        a[i] = (i + 1) * 10;

    printf("After realloc: ");
    for (i = 0; i < 10; i++)
        printf("%d ", a[i]);
    printf("\n");

    // free
    free(a);
    free(b);

    printf("Memory freed successfully.\n");

    return 0;
}
```

Output:

```
root@DUDDUKURIs  x  +  v  -  □  x

root@DUDDUKURIs:~$ cd ~/OS_Skill
mkdir -p Week-8/src Week-8/bin
cd Week-8
root@DUDDUKURIs:~/OS_Skill/Week-8$ nano src/dynamic_memory.c

root@DUDDUKURIs:~/OS_Skill/Week-8$ gcc src/dynamic_memory.c
-o bin/dynamic_memory
root@DUDDUKURIs:~/OS_Skill/Week-8$ ./bin/dynamic_memory
malloc: 10 20 30 40 50
calloc: 0 0 0 0 0
After realloc: 10 20 30 40 50 60 70 80 90 100
Memory freed successfully.
root@DUDDUKURIs:~/OS_Skill/Week-8$ valgrind --version
Command 'valgrind' not found, but can be installed with:
sudo snap install valgrind # version 3.26.0, or
sudo apt install valgrind # version 1:3.26.0-0ubuntu1
See 'snap info valgrind' for additional versions.
root@DUDDUKURIs:~/OS_Skill/Week-8$ sudo apt update
sudo apt install valgrind
```

```
root@DUDDUKURIs  x  +  v  -  □  x

root@DUDDUKURIs:~/OS_Skill/Week-8$ nano src/memory_leak.c
root@DUDDUKURIs:~/OS_Skill/Week-8$ gcc src/memory_leak.c -o
bin/memory_leak
root@DUDDUKURIs:~/OS_Skill/Week-8$ valgrind --leak-check=ful
l ./bin/memory_leak
==1978== Memcheck, a memory error detector
==1978== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et
al.
==1978== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyrigh
t info
==1978== Command: ./bin/memory_leak
==1978==
Memory allocated successfully.
==1978==
==1978== HEAP SUMMARY:
==1978==   in use at exit: 400 bytes in 1 blocks
==1978==   total heap usage: 2 allocs, 1 frees, 1,424 bytes allocated
==1978==
==1978== 400 bytes in 1 blocks are definitely lost in loss record 1 o
f 1
==1978==    at 0x4850858: malloc (vg_replace_malloc.c:447)
==1978==    by 0x400117E: main (in /home/priyanka/OS_Skill/Week-8/bin
/memory_leak)
==1978==
==1978== LEAK SUMMARY:
==1978==   definitely lost: 400 bytes in 1 blocks
==1978==   indirectly lost: 0 bytes in 0 blocks
==1978==   possibly lost: 0 bytes in 0 blocks
==1978==   still reachable: 0 bytes in 0 blocks
==1978==   suppressed: 0 bytes in 0 blocks
==1978==
==1978== For lists of detected and suppressed errors, rerun with: -s
==1978== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from
```

```
==1736== Memcheck, a memory error detector
==1736== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et
al.
==1736== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyrigh
t info
==1736== Command: ./bin/dynamic_memory
==1736==
malloc: 10 20 30 40 50
calloc: 0 0 0 0 0
After realloc: 10 20 30 40 50 60 70 80 90 100
Memory freed successfully.
==1736==
==1736== HEAP SUMMARY:
==1736==   in use at exit: 0 bytes in 0 blocks
==1736==   total heap usage: 4 allocs, 4 frees, 1,104 bytes allocated
==1736==
==1736== All heap blocks were freed -- no leaks are possible
==1736==
==1736== For lists of detected and suppressed errors, rerun with: -s
==1736== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from
0)
```

2. Develop a program that demonstrates Copy-on-Write behavior after a `fork()`. Analyze memory usage before and after modifying data in the child process.

```
root@DUDDUKURIs  ×  +  v
GNU nano 8.7.1 src/copy_on_write.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE 1000000

int main() {
    int *data = malloc(SIZE * sizeof(int));

    if (data == NULL) {
        perror("malloc");
        return 1;
    }

    for (int i = 0; i < SIZE; i++)
        data[i] = i;

    printf("Parent PID: %d\n", getpid());
    printf("Data initialized.\n");
    printf("Forking process...\n");

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        free(data);
        return 1;
    }

    if (pid == 0) {
        printf("\nChild PID: %d\n", getpid());
        printf("Child reads data: %d\n", data[0]);

        printf("Child modifying data...\n");

        printf("Child modifying data...\n");

        for (int i = 0; i < SIZE; i++)
            data[i] = -data[i];

        printf("Child modified data: %d\n", data[1]);

        sleep(10);

        free(data);
        exit(0);
    }
    else {
        printf("\nParent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
        printf("Parent data remains: %d\n", data[1]);

        sleep(10);

        wait(NULL);

        free(data);
    }

    return 0;
}
```

Output:

```
root@DUDDUKURIs:~/OS_Skill/Week-8$ nano src/copy_on_write.c
root@DUDDUKURIs:~/OS_Skill/Week-8$ gcc src/copy_on_write.c -o bin/
copy_on_write
root@DUDDUKURIs:~/OS_Skill/Week-8$ ./bin/copy_on_write
Parent PID: 2112
Data initialized.
Forking process...

Parent PID: 2112
Child PID: 2113
Parent data remains: 1

Child PID: 2113
Child reads data: 0
Child modifying data...
Child modified data: -1
```